

Lab-2 Assignment

Procedure

Accept the assignment

Submit all your code to your repository under Lab 2 folder by the due date time.

Objective

The objective here is to set up a Command-Line Interface (CLI) through which you can communicate with your target board. Such a tool is extremely useful for giving commands, unit testing your code, and for automating scripted unit tests.

First you will be enabling an onboard USART and establishing simple polled serial communication with a terminal program (e.g. Putty, or TeraTerm) running on the host machine. [If you have a MAC, you can use this link.](#)

Next, you will design a set of commands and responses such that you can type the commands at your CLI prompt, and expect to see various behaviours on the target board as well as textual responses inside the CLI. One useful exercise will incorporate LED control into your CLI.

At each step of the development, we will be paying attention to good software design principles.

Procedure

Phase 1: Separation of Student application code

Given the information in the lab lecture detailing how to separate your code from the generated code inside our IDE, load your previous lab and modify the project based on that framework.

Phase 2: Get the Serial Port Working

Project Setup

- Create a new Lab2 subdirectory inside your repository on your local machine.
- Start the STM32CubeIDE software and create a new STM32 project
- Search and Select our Nucleo-64 board.
- Initiate all peripherals with their default mode.
- Use the graphical tool to enable USART2. This is the USB connection between the board and the host. Notice in the schematic that this USART2 connects to the PC through the ST-Link. This allows us communication between the board and PC.
- Save the project and generate all the setup code.

There are a number of ways to communicate with the board via the serial link. The first way we will investigate is polling mode. Investigate the following two HAL functions found in 39.2.1 of the HAL document under Polling mode IO operation:

```
HAL_UART_Transmit()  
HAL_UART_Receive()
```

Notice that these two routines are Blocking routines. In other words no other code can run until these routines return. Investigate these routines and come up with a way to communicate between the target and host in efficient manner.

Phase 3: Implement CLI and various commands

The CLI should print some sort of prompt. Blank lines merely repeat the prompt. The user must be able to use the backspace key to correct typing errors. All commands must produce some textual output as confirmation the command worked, or an error message if the command was malformed. You should implement commands to do the following:

- Turn off or on the LED.
- Query the state of the LED.
- "help" should print a screen of help explaining the commands.

The CLI should be in its own files such as cli.c and cli.h. This section of code should implement the CLI entity.

Submission

Make sure you have committed your latest solution and pushed it to your repository for grading.